LLM Inference Optimizer

A Plain-English Overview

A guide anyone can follow - no technical background required.

1. The problem, in everyday terms

A modern AI language model (the kind behind chatbots) is like a brilliant but slow-to-respond expert. It knows a great deal, but every time you ask it something it must do an enormous amount of mental arithmetic to produce each word. On a computer that arithmetic is slow and expensive - it ties up costly graphics processors (GPUs) and makes you wait. Two questions matter to anyone running these models:

- How do we make the model answer faster and serve more people at once?
- How do we prove it actually got faster - with real numbers, not guesses?

This project answers both.

2. What this project is

It is an assembly line for taking a large AI model and making it run faster on a GPU, plus a measurement lab that times everything precisely so the speed-ups are proven, not claimed. Think of a workshop that tunes a fast-but-thirsty car engine for efficiency, then puts it on a dynamometer to measure the exact horsepower gain.

3. How it works - the six stations on the assembly line

Each stage takes the model one step closer to running faster; each feeds the next.

- Export - convert the model into a standard, portable format (ONNX) that optimization tools understand. Like translating a recipe into a universal format.
- Optimize - re-compile the model for the exact GPU (TensorRT), and shrink it by storing numbers with less precision (quantization). Like compressing a huge photo into a smaller file that still looks fine: ~3x smaller and faster.
- Serve - one unified way to run the model, with three interchangeable engines: eager (the standard baseline), ONNX (the portable build), and vLLM (a state-of-the-art server that handles many requests at once - the fast one).
- Profile - put a stopwatch and a power meter on the model: time to first word, speed of each following word, and how much electricity the GPU draws.
- Benchmark - run the model under many workloads (small vs large batches, short vs long inputs) and record every result to a spreadsheet automatically.
- Analyze - turn those spreadsheets into charts and a clear bottom line.

4. What was achieved

Everything was built and proven on a real GPU (a free Google Colab T4), end to end. Highlights from the measured results:

- ~2.3x more throughput and ~57% less waiting per word using the vLLM engine versus the standard baseline - identical hardware and model.

- ~24x more throughput from batching alone: serving 32 requests together instead of one at a time (28 -> 686 words per second).
- A model made ~3x smaller via quantization (2.2 GB shrunk to ~0.76 GB) while still running correctly.
- Real power and memory captured automatically (~45-67 watts under load) - so efficiency, not just speed, is quantified.

These numbers come from a small model on modest free hardware; the same pipeline is built to run a much larger model on a powerful GPU, where the gains are larger.

5. The hard part (and why it shows real skill)

Building the pipeline was only half the work. The other half was making it run on real machines, which fought back constantly:

- Mismatched software versions - different stages needed conflicting versions of a shared library. Fixed by giving each stage its own clean workspace.
- GPU driver mismatches - several libraries shipped builds for a newer GPU platform than the machine had, causing cryptic crashes. Each was pinned to the right build.
- A multi-layer puzzle to get the fast (vLLM) engine running - five separate issues unwound in sequence, then baked into the code so it self-heals.

Every one of these was found by actually running the system, fixed, and locked in with automated tests so it stays fixed.

6. How quality was kept high

- Automated tests run on every change, on an ordinary laptop, catching mistakes early.
- Every GPU-only piece was validated on a real GPU before being marked done.
- Every fix and decision is documented and version-controlled - fully reproducible.

7. Why it matters

Faster, cheaper AI inference means lower running costs, snappier responses, and serving more people with the same hardware. This project delivers a complete, measured, reproducible path to that - and the evidence to back it up.

Tech stack: PyTorch, CUDA, TensorRT, ONNX, vLLM, NVML. Status: end-to-end pipeline complete and validated on GPU.